C / C++

Java

Python

JavaScript

<DSA>
algorithm
data_structures
complexity_analysis

Data Structures & Algorithms

The Complete A to Z Guide

C / C++ | Java | Python | JavaScript

Mohammad Faiz

Year 2026 - Multi-Language Edition

01001000 01100101
01101100 01101100
01101111 00100000
01010111 01101111
01110010 01101100
01100100 00100001

Copyright Notice

Copyright (c) 2026 Mohammad Faiz. All Rights Reserved.

No Permission to Use: No person or entity is permitted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this content for any purpose (commercial or non-commercial).

Personal Use Only: Only the original author Mohammad Faiz has the authority to use this content in any project or application.

Attribution: If you discuss or reference the concepts, you must provide full credit to the original author Mohammad Faiz.

Multi-Language Edition

This book includes code examples in:

- C/C++
- Java
- Python
- JavaScript

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Chapter 1: Introduction to DSA.....1

1.1 What is Data Structure?.....1

1.2 What is Algorithm?.....2

Chapter 2: Arrays.....4

2.1 Introduction to Arrays.....4

2.2 Array Operations.....5

Chapter 3: Linked Lists.....10

3.1 Singly Linked List.....10

3.2 Doubly Linked List.....12

Chapter 4: Stacks.....16

4.1 Stack Implementation.....16

4.2 Stack Applications.....18

Chapter 5: Queues.....22

5.1 Types of Queues.....22

5.2 Queue Applications.....24

Chapter 6: Trees.....28

6.1 Binary Trees.....28

6.2 Binary Search Trees.....30

Chapter 7: Heaps.....36

Chapter 8: Graphs.....40

Chapter 9: Hashing.....48

Chapter 10: Sorting Algorithms.....54

Chapter 11: Searching Algorithms.....66

Chapter 12: Dynamic Programming.....72

Chapter 13: Greedy Algorithms.....80

Chapter 14: Backtracking.....86

Chapter 15: Advanced Data Structures.....92

Chapter 16: Interview Preparation.....100

Appendix A: Language Syntax Reference.....110

Appendix B: Cheat Sheet.....115

Chapter 1: Introduction to Data Structures and Algorithms

Welcome to the world of Data Structures and Algorithms! This chapter will introduce you to the fundamental concepts that form the backbone of computer science and programming. Whether you are a beginner or looking to strengthen your foundations, this book will guide you through every concept with simple explanations and practical examples.

1.1 What is Data Structure?

A Data Structure is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. Think of it like organizing your room - if you throw everything randomly, it takes forever to find what you need. But if you organize things properly, you can find anything quickly.

Real-Life Example

Imagine a library. Books are not organized randomly on shelves. They are organized by categories, authors, and titles. This organization (data structure) helps you find any book quickly. Similarly, in programming, we organize data so computers can find and use it efficiently.

Types of Data Structures

- Linear Data Structures: Arrays, Linked Lists, Stacks, Queues
- Non-Linear Data Structures: Trees, Graphs
- Hash-based: Hash Tables, Hash Maps

1.2 What is Algorithm?

An Algorithm is a step-by-step set of instructions to solve a specific problem. It is like a recipe for cooking - you follow each step in order to get the desired result. In programming, algorithms tell the computer exactly what to do.

Real-Life Example

Making a cup of tea is an algorithm:

0. Boil water
1. Put tea bag in cup
2. Pour hot water

3. Wait for 3 minutes
4. Remove tea bag and enjoy

1.3 Why Learn DSA?

Learning Data Structures and Algorithms is essential because:

- Efficiency: Write faster and more efficient programs
- Problem Solving: Develop logical thinking and problem-solving skills
- Interviews: Most tech companies test DSA knowledge in interviews
- Foundation: Essential for advanced computer science topics

1.4 Time and Space Complexity

When we talk about algorithms, we need to measure how good they are. We use two main measures:

Time Complexity

Time complexity tells us how much time an algorithm takes to run as the input size grows. We use Big O notation to express this.

Common Time Complexities

- O(1) - Constant Time: Same time regardless of input size
- O(log n) - Logarithmic: Very efficient, like binary search
- O(n) - Linear: Time grows proportionally with input
- O(n log n) - Linearithmic: Efficient sorting algorithms
- $O(n^2)$ - Quadratic: Nested loops, slower
- $O(2^n)$ - Exponential: Very slow, avoid if possible

Space Complexity

Space complexity tells us how much memory an algorithm uses. Just like time, we want to use as little memory as possible.

1.5 Practice Questions

Question 1: What is the time complexity of accessing an array element by index?

Answer: O(1) - Constant time. Arrays store elements in contiguous memory, so we can calculate the exact address directly.

Question 2: Why are data structures important?

Answer: Data structures help organize data efficiently, making operations like search, insert, and delete faster and more memory-efficient.

Question 3: What is the difference between time and space complexity?

Answer: Time complexity measures how fast an algorithm runs, while space complexity measures how much memory it uses.

Chapter 2: Arrays

Arrays are the simplest and most fundamental data structure. They store elements of the same type in contiguous memory locations, making them very efficient for certain operations.

2.1 Introduction to Arrays

An array is a collection of items stored at contiguous memory locations. All elements in an array are of the same data type (like all integers or all characters).

Real-Life Example

Think of a row of lockers in a school. Each locker has a number (index), and you can directly go to any locker if you know its number. This is exactly how arrays work!

Array Declaration - Multi-Language

C++

```
// C++ Array Declaration
int numbers[5] = {10, 20, 30, 40, 50};
string names[3] = {"Alice", "Bob", "Charlie"};
```

Java

```
// Java Array Declaration
int[] numbers = {10, 20, 30, 40, 50};
String[] names = {"Alice", "Bob", "Charlie"};
```

Python

```
# Python List (similar to array)
numbers = [10, 20, 30, 40, 50]
names = ["Alice", "Bob", "Charlie"]
```

2.2 Array Operations

1. Accessing Elements

Array elements are accessed using their index. In most programming languages, array indices start at 0.

C++

```
int firstElement = numbers[0];  // Gets 10
```

```
int thirdElement = numbers[2];  // Gets 30
```

Java

```
int firstElement = numbers[0];  // Gets 10
int thirdElement = numbers[2];  // Gets 30
```

Python

```
first_element = numbers[0]   # Gets 10
third_element = numbers[2]   # Gets 30
```

2. Inserting Elements

Inserting at the end is O(1), but inserting in the middle requires shifting elements, making it O(n).

C++

```
// Insert 25 at index 2
for (int i = 4; i >= 2; i--) {
    numbers[i+1] = numbers[i];
}
numbers[2] = 25;
```

Python

```
# Insert 25 at index 2
numbers.insert(2, 25)  # Simple and efficient!
```

3. Traversing an Array

C++

```
// C++ - For loop
for (int i = 0; i < 5; i++) {
    cout << numbers[i] << " ";
}

// C++ - Range-based for loop (C++11+)
for (int num : numbers) {
    cout << num << " ";
}
```

Java

```
// Java - For loop
for (int i = 0; i < numbers.length; i++) {
    System.out.print(numbers[i] + " ");
}
```

```
// Java - Enhanced for loop
for (int num : numbers) {
    System.out.print(num + " ");
}
```

Python

```
# Python - For loop
for num in numbers:
    print(num, end=" ")

# Python - List comprehension
[print(num, end=" ") for num in numbers]
```

2.3 Types of Arrays

1. One-Dimensional Arrays

A simple list of elements, like a single row of lockers.

2. Two-Dimensional Arrays

Like a table with rows and columns, or a matrix in mathematics.

C++

```
int matrix[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Java

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Python

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

2.4 Advantages and Disadvantages

Advantages

- Fast access: O(1) time to access any element by index
- Memory efficient: No extra memory for pointers
- Cache friendly: Contiguous memory improves performance

Disadvantages

- Fixed size: Size must be known at declaration (in most languages)
- Expensive insertions/deletions: O(n) for middle operations
- Same data type: Cannot mix different types

2.5 Practice Questions

Question 1: Find the maximum element in an array

C++

```
int findMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
// Time Complexity: O(n)
```

Java

```
int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) max = arr[i];
    }
    return max;
}
// Time Complexity: O(n)
```

Python

```
def find_max(arr):
    return max(arr)  # Built-in function
```

```
# Or manually:
def find_max_manual(arr):
    max_val = arr[0]
    for num in arr[1:]:
        if num > max_val:
            max_val = num
    return max_val
# Time Complexity: O(n)
```

Question 2: Reverse an array

C++

```
void reverse(int arr[], int n) {
    int start = 0, end = n - 1;
    while (start < end) {
        swap(arr[start], arr[end]);
        start++; end--;
    }
}
```

Python

```
# Python - Multiple ways
arr.reverse()           # In-place
reversed_arr = arr[::-1]  # Returns new list
```

Interview Question: What is the time complexity of array insertion at the beginning?

Answer: O(n) because all existing elements need to be shifted one position to the right.

Chapter 3: Linked Lists

Linked Lists are linear data structures where elements are stored in nodes, and each node points to the next node. Unlike arrays, elements are not stored in contiguous memory locations.

3.1 Singly Linked List

In a singly linked list, each node contains data and a pointer to the next node. The last node points to NULL.

Node Structure - Multi-Language

C++

```
struct Node {
    int data;
    Node* next;
};
```

Java

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Python

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Real-Life Example

Think of a treasure hunt. Each clue (node) tells you where to find the next clue (next pointer). You follow the chain until you reach the treasure (NULL).

Inserting at the Beginning

C++

```
void insertAtBeginning(Node*& head, int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = head;
    head = newNode;
}
// Time Complexity: O(1)
```

Java

```
void insertAtBeginning(Node head, int data) {
    Node newNode = new Node(data);
    newNode.next = head;
    head = newNode;
}
// Time Complexity: O(1)
```

Python

```
def insert_at_beginning(head, data):
    new_node = Node(data)
    new_node.next = head
    return new_node  # Return new head
# Time Complexity: O(1)
```

3.2 Doubly Linked List

In a doubly linked list, each node has two pointers - one to the next node and one to the previous node. This allows traversal in both directions.

Node Structure

C++

```
struct DNode {
    int data;
    DNode* next;
    DNode* prev;
};
```

Python

```
class DNode:
    def __init__(self, data):
```

```
        self.data = data
        self.next = None
        self.prev = None
```

Advantages of Doubly Linked List

- Can traverse in both directions
- Easy to delete a node if we have its pointer
- Can insert before a node easily

3.3 Linked List vs Array

Aspect	Array	Linked List
Memory	Contiguous	Non-contiguous
Access	O(1)	O(n)
Insertion	O(n)	O(1)
Deletion	O(n)	O(1)
Size	Fixed	Dynamic

3.4 Practice Questions

Question 1: Reverse a Linked List

C++

```
Node* reverse(Node* head) {
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;

    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
// Time Complexity: O(n)
```

Python

```python
def reverse(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    return prev
# Time Complexity: O(n)
```

Question 2: Detect Loop in Linked List (Floyd's Cycle)

C++

```cpp
bool hasCycle(Node* head) {
    if (head == NULL) return false;
    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
// Time: O(n), Space: O(1)
```

Python

```python
def has_cycle(head):
    if not head: return False
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
# Time: O(n), Space: O(1)
```

Interview Question: Why use linked lists over arrays?

Answer: Linked lists are better when you need frequent insertions/deletions, especially at the beginning or middle, as they don't require shifting elements like arrays do.

Chapter 4: Stacks

A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. The last element added is the first one to be removed.

4.1 Stack Implementation

Real-Life Example

Think of a stack of plates. You can only add a new plate on top, and you can only remove the top plate. You cannot take a plate from the middle without removing the plates above it first.

Stack Operations

- Push: Add an element to the top of the stack
- Pop: Remove and return the top element
- Peek/Top: View the top element without removing it
- isEmpty: Check if the stack is empty

Array-based Implementation - C++

```
class Stack {
    int top;
    int capacity;
    int* array;

public:
    Stack(int size) {
        capacity = size;
        array = new int[capacity];
        top = -1;
    }

    void push(int x) {
        if (top >= capacity - 1) {
            cout << "Stack Overflow";
            return;
        }
        array[++top] = x;
    }
```

```
    int pop() {
        if (top < 0) {
            cout << "Stack Underflow";
            return -1;
        }
        return array[top--];
    }

    int peek() {
        if (top < 0) return -1;
        return array[top];
    }

    bool isEmpty() {
        return top < 0;
    }
};
```

Using Built-in Stack - Java

```
import java.util.Stack;

Stack<Integer> stack = new Stack<>();
stack.push(10);
stack.push(20);
int top = stack.peek();  // 20
int popped = stack.pop();  // 20
boolean empty = stack.isEmpty();  // false
```

Using List as Stack - Python

```
# Python list works as a stack
stack = []
stack.append(10)   # push
stack.append(20)   # push
top = stack[-1]    # peek
popped = stack.pop()  # pop

# Or use collections.deque for better performance
from collections import deque
stack = deque()
stack.append(10)
stack.append(20)
```

```
popped = stack.pop()
```

4.2 Stack Applications

1. Balanced Parentheses Check

C++

```
bool isBalanced(string expr) {
    Stack s;
    for (char c : expr) {
        if (c == '(' || c == '[' || c == '{')
            s.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (s.isEmpty()) return false;
            char top = s.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{'))
                return false;
        }
    }
    return s.isEmpty();
}
```

Python

```
def is_balanced(expr):
    stack = []
    mapping = {')': '(', ']': '[', '}': '{'}
    for char in expr:
        if char in mapping.values():
            stack.append(char)
        elif char in mapping.keys():
            if not stack or stack.pop() != mapping[char]:
                return False
    return not stack
```

4.3 Practice Questions

Question: Next Greater Element

C++

```
vector<int> nextGreater(vector<int>& arr) {
```

```
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> s;

    for (int i = 0; i < n; i++) {
        while (!s.empty() && arr[s.top()] < arr[i]) {
            result[s.top()] = arr[i];
            s.pop();
        }
        s.push(i);
    }
    return result;
}
```

Python

```
def next_greater(arr):
    n = len(arr)
    result = [-1] * n
    stack = []

    for i in range(n):
        while stack and arr[stack[-1]] < arr[i]:
            result[stack.pop()] = arr[i]
        stack.append(i)
    return result
```

Interview Question: What is stack overflow?

Answer: Stack overflow occurs when we try to push an element onto a full stack (in array implementation) or when the recursion depth exceeds the system stack limit.

Chapter 5: Queues

A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. The first element added is the first one to be removed.

5.1 Types of Queues

Real-Life Example

Think of a line at a ticket counter. The person who arrives first gets served first. This is exactly how queues work in programming.

1. Simple Queue

Basic queue with enqueue (add to rear) and dequeue (remove from front) operations.

C++

```
class Queue {
    int front, rear, size;
    int capacity;
    int* array;

public:
    Queue(int cap) {
        capacity = cap;
        front = size = 0;
        rear = capacity - 1;
        array = new int[capacity];
    }

    void enqueue(int item) {
        if (isFull()) return;
        rear = (rear + 1) % capacity;
        array[rear] = item;
        size++;
    }

    int dequeue() {
        if (isEmpty()) return -1;
        int item = array[front];
        front = (front + 1) % capacity;
```

```
        size--;
        return item;
    }

    bool isFull() { return size == capacity; }
    bool isEmpty() { return size == 0; }
};
```

Java

```
import java.util.LinkedList;
import java.util.Queue;

Queue<Integer> queue = new LinkedList<>();
queue.offer(10);   // enqueue
queue.offer(20);
int front = queue.peek();  // view front
int item = queue.poll();   // dequeue
```

Python

```
from collections import deque

queue = deque()
queue.append(10)    # enqueue
queue.append(20)
front = queue[0]    # peek
item = queue.popleft()  # dequeue

# Or use SimpleQueue for thread-safe operations
from queue import SimpleQueue
q = SimpleQueue()
q.put(10)
item = q.get()
```

2. Circular Queue

In a circular queue, the last position is connected back to the first position, forming a circle. This efficiently uses space.

3. Priority Queue

Elements are served based on priority, not just arrival order.

Python

```
import heapq
```

```
# Min heap by default
pq = []
heapq.heappush(pq, 3)
heapq.heappush(pq, 1)
heapq.heappush(pq, 2)
smallest = heapq.heappop(pq)  # Returns 1
```

5.2 Implementing Queue using Two Stacks

C++

```
class QueueUsingStacks {
    stack<int> s1, s2;

public:
    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if (s2.empty()) {
            if (s1.empty()) return -1;
            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        int x = s2.top();
        s2.pop();
        return x;
    }
};
```

Python

```
class QueueUsingStacks:
    def __init__(self):
        self.s1 = []
        self.s2 = []

    def enqueue(self, x):
        self.s1.append(x)
```

```
    def dequeue(self):
        if not self.s2:
            if not self.s1:
                return None
            while self.s1:
                self.s2.append(self.s1.pop())
        return self.s2.pop()
```

5.3 Practice Questions

Question: Generate Binary Numbers from 1 to N

C++

```
vector<string> generateBinary(int n) {
    vector<string> result;
    queue<string> q;
    q.push("1");

    for (int i = 0; i < n; i++) {
        string curr = q.front();
        q.pop();
        result.push_back(curr);
        q.push(curr + "0");
        q.push(curr + "1");
    }
    return result;
}
```

Python

```
from collections import deque

def generate_binary(n):
    result = []
    queue = deque(["1"])

    for _ in range(n):
        curr = queue.popleft()
        result.append(curr)
        queue.append(curr + "0")
        queue.append(curr + "1")
    return result
```

Interview Question: Difference between Stack and Queue?

Answer: Stack follows LIFO (Last-In-First-Out) where the last element added is removed first. Queue follows FIFO (First-In-First-Out) where the first element added is removed first.

Chapter 6: Trees

A Tree is a hierarchical data structure consisting of nodes connected by edges. It starts with a root node and each node can have child nodes.

6.1 Binary Trees

A Binary Tree is a tree where each node has at most two children, referred to as left child and right child.

Node Structure - Multi-Language

C++

```
struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int val) {
        data = val;
        left = right = NULL;
    }
};
```

Java

```
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        data = val;
        left = right = null;
    }
}
```

Python

```
class TreeNode:
    def __init__(self, val=0):
        self.data = val
        self.left = None
```

```
        self.right = None
```

Tree Traversals

Tree traversal means visiting all nodes of a tree in a specific order.

Inorder Traversal (Left-Root-Right)

C++

```
void inorder(TreeNode* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

Python

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
```

Level Order Traversal (BFS)

C++

```
void levelOrder(TreeNode* root) {
    if (root == NULL) return;
    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* curr = q.front();
        q.pop();
        cout << curr->data << " ";
        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}
```

Python

```
from collections import deque
```

```python
def level_order(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        curr = queue.popleft()
        print(curr.data, end=" ")
        if curr.left:
            queue.append(curr.left)
        if curr.right:
            queue.append(curr.right)
```

6.2 Binary Search Trees (BST)

A Binary Search Tree is a binary tree where for each node:

- All values in the left subtree are smaller than the node's value
- All values in the right subtree are greater than the node's value
- Both left and right subtrees are also BSTs

Search in BST

C++

```cpp
TreeNode* search(TreeNode* root, int key) {
    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)
        return search(root->left, key);
    return search(root->right, key);
}
// Time Complexity: O(h) where h is height
```

Python

```python
def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
    return search(root.right, key)
# Time Complexity: O(h) where h is height
```

6.3 Practice Questions

Question: Find Height of Binary Tree

C++

```
int height(TreeNode* root) {
    if (root == NULL) return -1;
    return 1 + max(height(root->left), height(root->right));
}
```

Python

```
def height(root):
    if root is None:
        return -1
    return 1 + max(height(root.left), height(root.right))
```

Question: Validate BST

Python

```
def is_valid_bst(root, min_val=float('-inf'), max_val=float('inf')):
    if not root:
        return True
    if root.data <= min_val or root.data >= max_val:
        return False
    return (is_valid_bst(root.left, min_val, root.data) and
            is_valid_bst(root.right, root.data, max_val))
```

Interview Question: Why use BST over regular binary tree?

Answer: BST provides O(log n) time complexity for search, insert, and delete operations in balanced cases, whereas a regular binary tree may require O(n) time.

Chapter 7: Heaps and Priority Queues

A Heap is a special tree-based data structure that satisfies the heap property. It is commonly used to implement priority queues.

7.1 Types of Heaps

1. Max Heap

In a Max Heap, the value of each parent node is greater than or equal to the values of its children. The root contains the maximum value.

2. Min Heap

In a Min Heap, the value of each parent node is less than or equal to the values of its children. The root contains the minimum value.

7.2 Heap Operations

Using Priority Queue - Java

```
import java.util.PriorityQueue;

// Min Heap (default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.offer(5);
minHeap.offer(1);
minHeap.offer(3);
int min = minHeap.poll();  // Returns 1

// Max Heap
PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
maxHeap.offer(5);
maxHeap.offer(1);
int max = maxHeap.poll();  // Returns 5
```

Using heapq - Python

```
import heapq
```

```
# Min heap (default)
min_heap = []
heapq.heappush(min_heap, 5)
heapq.heappush(min_heap, 1)
heapq.heappush(min_heap, 3)
min_val = heapq.heappop(min_heap)  # Returns 1

# Max heap (store negatives)
max_heap = []
heapq.heappush(max_heap, -5)
heapq.heappush(max_heap, -1)
max_val = -heapq.heappop(max_heap)  # Returns 5
```

7.3 Applications of Heaps

- Priority Queues: Efficiently get the highest/lowest priority element
- Heap Sort: O(n log n) sorting algorithm
- Graph Algorithms: Dijkstra's and Prim's algorithms
- Finding Kth Largest/Smallest Element

7.4 Practice Questions

Question: Find Kth Largest Element

C++

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;

    for (int num : nums) {
        minHeap.push(num);
        if (minHeap.size() > k)
            minHeap.pop();
    }
    return minHeap.top();
}
// Time: O(n log k), Space: O(k)
```

Python

```
import heapq

def find_kth_largest(nums, k):
```

```
    # Use min heap of size k
    min_heap = nums[:k]
    heapq.heapify(min_heap)

    for num in nums[k:]:
        if num > min_heap[0]:
            heapq.heapreplace(min_heap, num)

    return min_heap[0]
# Time: O(n log k), Space: O(k)
```

Interview Question: When would you use a heap over a sorted array?

Answer: Use a heap when you need to frequently insert elements and extract min/max. Heap operations are O(log n) while maintaining sorted order in an array would be O(n) for insertions.

Chapter 8: Graphs

A Graph is a non-linear data structure consisting of vertices (nodes) connected by edges. Graphs are used to represent networks, relationships, and connections.

8.1 Graph Terminology

- Vertex (Node): A fundamental unit of a graph
- Edge: A connection between two vertices
- Directed Graph: Edges have direction (one-way)
- Undirected Graph: Edges have no direction (two-way)
- Weighted Graph: Edges have associated weights/costs

8.2 Graph Representation

Adjacency List - Multi-Language

C++

```
// Adjacency List
vector<int> adjList[V];

void addEdge(int u, int v) {
    adjList[u].push_back(v);
    adjList[v].push_back(u);  // For undirected
}
// Space: O(V + E)
```

Java

```
// Adjacency List
List<List<Integer>> adjList = new ArrayList<>();
for (int i = 0; i < V; i++)
    adjList.add(new ArrayList<>());

void addEdge(int u, int v) {
    adjList.get(u).add(v);
    adjList.get(v).add(u);  // For undirected
}
```

Python

```python
# Adjacency List
from collections import defaultdict

adj_list = defaultdict(list)

def add_edge(u, v):
    adj_list[u].append(v)
    adj_list[v].append(u)  # For undirected

# Or use dictionary
graph = {
    0: [1, 2],
    1: [0, 2],
    2: [0, 1]
}
```

8.3 Graph Traversals

Breadth-First Search (BFS)

C++

```cpp
void BFS(int start, vector<int> adj[], int V) {
    vector<bool> visited(V, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
// Time: O(V + E), Space: O(V)
```

Python

```python
from collections import deque

def bfs(start, adj, V):
    visited = [False] * V
    queue = deque([start])
    visited[start] = True

    while queue:
        u = queue.popleft()
        print(u, end=" ")

        for v in adj[u]:
            if not visited[v]:
                visited[v] = True
                queue.append(v)
# Time: O(V + E), Space: O(V)
```

Depth-First Search (DFS)

Python

```python
def dfs(u, adj, visited):
    visited[u] = True
    print(u, end=" ")

    for v in adj[u]:
        if not visited[v]:
            dfs(v, adj, visited)

def dfs_util(V, adj):
    visited = [False] * V
    for i in range(V):
        if not visited[i]:
            dfs(i, adj, visited)
```

8.4 Dijkstra's Algorithm (Shortest Path)

Python

```python
import heapq

def dijkstra(graph, src, V):
    dist = [float('inf')] * V
```

```
    dist[src] = 0
    pq = [(0, src)]

    while pq:
        d, u = heapq.heappop(pq)

        if d > dist[u]:
            continue

        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))

    return dist
# Time: O((V + E) log V)
```

8.5 Practice Questions

Question: Number of Islands

Python

```
def num_islands(grid):
    if not grid:
        return 0

    rows, cols = len(grid), len(grid[0])
    count = 0

    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        dfs(r+1, c)
        dfs(r-1, c)
        dfs(r, c+1)
        dfs(r, c-1)

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                dfs(r, c)
```

```
                count += 1

    return count
```

Interview Question: When to use BFS vs DFS?

Answer: Use BFS for finding shortest paths in unweighted graphs and when the solution is near the root. Use DFS for exploring all paths, detecting cycles, and when the solution is deep in the tree.

Chapter 9: Hashing and Hash Tables

Hashing is a technique that converts a large input (key) into a smaller, fixed-size value (hash) using a hash function. Hash tables use this technique for fast data retrieval.

9.1 Hash Function

A hash function maps keys to indices in a hash table. A good hash function should be fast to compute, distribute keys uniformly, and minimize collisions.

9.2 Using Hash Maps - Multi-Language

C++

```cpp
#include <unordered_map>

unordered_map<string, int> map;
map["apple"] = 5;
map["banana"] = 3;

int count = map["apple"];  // Returns 5

// Check if key exists
if (map.find("orange") != map.end()) {
    // Key exists
}

// Iterate
for (auto& pair : map) {
    cout << pair.first << ": " << pair.second << endl;
}
```

Java

```java
import java.util.HashMap;

HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 5);
map.put("banana", 3);

int count = map.get("apple");  // Returns 5
```

```
// Check if key exists
if (map.containsKey("orange")) {
    // Key exists
}

// Iterate
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

Python

```
# Dictionary (hash map)
map = {}
map["apple"] = 5
map["banana"] = 3

count = map["apple"]  # Returns 5

# Check if key exists
if "orange" in map:
    # Key exists
    pass

# Get with default
count = map.get("orange", 0)  # Returns 0 if not found

# Iterate
for key, value in map.items():
    print(f"{key}: {value}")

# Using Counter for frequency
from collections import Counter
freq = Counter([1, 2, 2, 3, 3, 3])  # {3: 3, 2: 2, 1: 1}
```

9.3 Applications of Hashing

- Database Indexing: Fast data retrieval
- Caching: Storing frequently accessed data
- Password Verification: Store hash instead of actual password
- Counting Frequencies: Count occurrences of elements

9.4 Practice Questions

Question: Two Sum

C++

```cpp
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> map;

    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (map.find(complement) != map.end())
            return {map[complement], i};
        map[nums[i]] = i;
    }
    return {};
}
// Time: O(n), Space: O(n)
```

Python

```python
def two_sum(nums, target):
    map = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in map:
            return [map[complement], i]
        map[num] = i
    return []
# Time: O(n), Space: O(n)
```

Question: Group Anagrams

Python

```python
from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)

    for s in strs:
        key = ''.join(sorted(s))
        groups[key].append(s)

    return list(groups.values())
```

Interview Question: What makes a good hash function?

Answer: A good hash function should be deterministic (same input always gives same output), fast to compute, distribute keys uniformly to minimize collisions, and use all input data.

Chapter 10: Sorting Algorithms

Sorting is arranging data in a particular order (ascending or descending). It is one of the most fundamental operations in computer science.

10.1 Built-in Sorting - Multi-Language

C++

```
#include <algorithm>

vector<int> arr = {5, 2, 8, 1, 9};

// Ascending order
sort(arr.begin(), arr.end());

// Descending order
sort(arr.begin(), arr.end(), greater<int>());

// Custom comparator
sort(arr.begin(), arr.end(), [](int a, int b) {
    return a > b;  // Descending
});
```

Java

```
import java.util.Arrays;
import java.util.Collections;

Integer[] arr = {5, 2, 8, 1, 9};

// Ascending order
Arrays.sort(arr);

// Descending order
Arrays.sort(arr, Collections.reverseOrder());
```

Python

```
arr = [5, 2, 8, 1, 9]

# Sort in-place (ascending)
arr.sort()
```

```
# Return new sorted list
sorted_arr = sorted(arr)

# Descending order
arr.sort(reverse=True)

# Sort by custom key
words = ["banana", "pie", "Washington"]
words.sort(key=len)  # Sort by length
```

10.2 Merge Sort

Divide and conquer algorithm that divides the array into halves, sorts them, and merges them back.

Python

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Time: O(n log n), Space: O(n)
```

10.3 Quick Sort

Python

```python
def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)
# Time: O(n log n) average, O(n²) worst
```

10.4 Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)

10.5 Practice Questions

Question: Sort an array of 0s, 1s, and 2s (Dutch National Flag)

Python

```python
def sort_colors(nums):
    low = mid = 0
    high = len(nums) - 1

    while mid <= high:
        if nums[mid] == 0:
```

```
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
# Time: O(n), Space: O(1)
```

Interview Question: When would you use QuickSort over MergeSort?

Answer: Use QuickSort when memory is limited (O(log n) space vs O(n) for MergeSort) and when average-case performance matters. However, MergeSort is preferred for linked lists and when stable sorting is needed.

Chapter 11: Searching Algorithms

Searching is the process of finding a specific element in a data structure. Efficient searching is crucial for many applications.

11.1 Linear Search

Python

```python
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1
# Time: O(n), Space: O(1)
```

11.2 Binary Search

Binary search works on sorted arrays by repeatedly dividing the search interval in half.

C++

```cpp
int binarySearch(int arr[], int n, int target) {
    int left = 0, right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;
        if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
// Time: O(log n), Space: O(1)
```

Python

```python
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
```

```
    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Or use bisect module
import bisect
index = bisect.bisect_left(arr, target)
```

11.3 Search in 2D Matrix

Python

```
def search_matrix(matrix, target):
    if not matrix:
        return False

    m, n = len(matrix), len(matrix[0])
    left, right = 0, m * n - 1

    while left <= right:
        mid = (left + right) // 2
        mid_val = matrix[mid // n][mid % n]

        if mid_val == target:
            return True
        elif mid_val < target:
            left = mid + 1
        else:
            right = mid - 1

    return False
```

11.4 Practice Questions

Question: Find Peak Element

Python

```python
def find_peak_element(nums):
    left, right = 0, len(nums) - 1

    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[mid + 1]:
            right = mid
        else:
            left = mid + 1

    return left
```

Interview Question: Why is binary search O(log n)?

Answer: Binary search halves the search space in each iteration. Starting with n elements, after k iterations we have n/(2^k) elements. We stop when n/(2^k) = 1, which means k = log n.

Chapter 12: Dynamic Programming

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into simpler subproblems, solving each subproblem only once, and storing their solutions.

12.1 Key Concepts

- Optimal Substructure: Optimal solution can be constructed from optimal solutions of subproblems
- Overlapping Subproblems: Same subproblems are solved multiple times

12.2 Fibonacci Numbers

Python

```
# Memoization (Top-Down)
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

# Tabulation (Bottom-Up)
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Space Optimized
def fib_opt(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for _ in range(2, n + 1):
        curr = prev1 + prev2
```

```
        prev2 = prev1
        prev1 = curr
    return prev1
```

12.3 0/1 Knapsack Problem

Python

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, W + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][W]
# Time: O(n*W), Space: O(n*W)
```

12.4 Longest Common Subsequence (LCS)

Python

```
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]
# Time: O(m*n), Space: O(m*n)
```

12.5 Practice Questions

Question: Climbing Stairs

Python

```python
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        curr = prev1 + prev2
        prev2 = prev1
        prev1 = curr
    return prev1
# Fibonacci pattern - Time: O(n), Space: O(1)
```

Question: Maximum Subarray (Kadane's Algorithm)

Python

```python
def max_sub_array(nums):
    max_so_far = max_ending_here = nums[0]

    for num in nums[1:]:
        max_ending_here = max(num, max_ending_here + num)
        max_so_far = max(max_so_far, max_ending_here)

    return max_so_far
# Time: O(n), Space: O(1)
```

Interview Question: When to use DP?

Answer: Use DP when the problem has optimal substructure and overlapping subproblems. Look for problems involving optimization (max/min), counting, or checking existence/possibility.

Chapter 13: Greedy Algorithms

Greedy algorithms make the locally optimal choice at each step with the hope of finding a global optimum. They are simple and efficient but do not always produce the optimal solution.

13.1 Key Characteristics

- Greedy Choice Property: A global optimum can be reached by choosing local optima
- Optimal Substructure: An optimal solution contains optimal solutions to subproblems

13.2 Activity Selection Problem

Python

```
def activity_selection(activities):
    # Sort by finish time
    activities.sort(key=lambda x: x[1])

    count = 1
    last_finish = activities[0][1]

    for i in range(1, len(activities)):
        if activities[i][0] >= last_finish:
            count += 1
            last_finish = activities[i][1]

    return count

# activities = [(start, finish), ...]
# Example: activities = [(1, 4), (3, 5), (0, 6), (5, 7)]
```

13.3 Fractional Knapsack

Python

```
def fractional_knapsack(weights, values, W):
    n = len(weights)
    items = [(values[i]/weights[i], weights[i], values[i])
             for i in range(n)]

    # Sort by value/weight ratio descending
```

```
    items.sort(reverse=True)

    max_value = 0
    for ratio, wt, val in items:
        if wt <= W:
            max_value += val
            W -= wt
        else:
            max_value += ratio * W
            break

    return max_value
```

13.4 Greedy vs Dynamic Programming

Aspect	Greedy	Dynamic Programming
Approach	Local optimal choice	Global optimal solution
Guarantee	May not be optimal	Always optimal
Speed	Faster	Slower
Examples	Dijkstra, Huffman	Knapsack, LCS

13.5 Practice Questions

Question: Jump Game

Python

```
def can_jump(nums):
    max_reach = 0
    for i in range(len(nums)):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + nums[i])
    return True
```

Interview Question: When does greedy fail?

Answer: Greedy fails when local optimal choices don't lead to global optimum. Example: 0/1 Knapsack cannot be solved greedily because we cannot take fractions of items - we need DP for optimal solution.

Chapter 14: Backtracking

Backtracking is a systematic way of trying out different sequences of decisions until a solution is found. When a path doesn't lead to a solution, we backtrack and try another path.

14.1 N-Queens Problem

Python

```python
def solve_n_queens(n):
    def is_safe(board, row, col):
        # Check column
        for i in range(row):
            if board[i][col] == 'Q':
                return False
        # Check diagonals
        for i, j in zip(range(row, -1, -1), range(col, -1, -1)):
            if board[i][j] == 'Q':
                return False
        for i, j in zip(range(row, -1, -1), range(col, n)):
            if board[i][j] == 'Q':
                return False
        return True

    def solve(board, row):
        if row == n:
            result.append([''.join(r) for r in board])
            return

        for col in range(n):
            if is_safe(board, row, col):
                board[row][col] = 'Q'
                solve(board, row + 1)
                board[row][col] = '.'  # Backtrack

    result = []
    board = [['.' for _ in range(n)] for _ in range(n)]
    solve(board, 0)
    return result
```

14.2 Permutations

Python

```python
def permute(nums):
    def backtrack(start):
        if start == len(nums):
            result.append(nums[:])
            return

        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]  # Backtrack

    result = []
    backtrack(0)
    return result
# Time: O(n!)
```

14.3 Subsets

Python

```python
def subsets(nums):
    def backtrack(start, current):
        result.append(current[:])

        for i in range(start, len(nums)):
            current.append(nums[i])
            backtrack(i + 1, current)
            current.pop()  # Backtrack

    result = []
    backtrack(0, [])
    return result
# Generate all subsets (power set)
```

14.4 Practice Questions

Question: Combination Sum

Python

```python
def combination_sum(candidates, target):
```

```
    def backtrack(remain, start, current):
        if remain == 0:
            result.append(current[:])
            return

        for i in range(start, len(candidates)):
            if candidates[i] <= remain:
                current.append(candidates[i])
                backtrack(remain - candidates[i], i, current)
                current.pop()

    result = []
    backtrack(target, 0, [])
    return result
```

Interview Question: How to optimize backtracking?

Answer: Use pruning to eliminate invalid paths early, memoization to avoid recomputing states, and heuristics to explore promising paths first. Also consider iterative approaches or dynamic programming when applicable.

Chapter 15: Advanced Data Structures

This chapter covers advanced data structures that are essential for solving complex problems efficiently.

15.1 Tries (Prefix Trees)

A Trie is a tree-like data structure used to store a dynamic set of strings, where the keys are usually strings.

Python

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word):
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end_of_word

    def starts_with(self, prefix):
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
```

```
            node = node.children[char]
        return True
```

Applications of Trie

- Auto-complete and spell checking
- IP routing (Longest prefix matching)
- Pattern matching

15.2 Union-Find (Disjoint Set)

Python

```python
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # Path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return

        # Union by rank
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1

    def connected(self, x, y):
        return self.find(x) == self.find(y)
# Amortized Time: O(alpha(n)) ~= O(1)
```

15.3 LRU Cache

Python

```python
from collections import OrderedDict
```

```
class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
# Time: O(1) for both get and put
```

15.4 Practice Questions

Question: Number of Provinces (Union-Find)

Python

```
def find_circle_num(is_connected):
    n = len(is_connected)
    uf = UnionFind(n)

    for i in range(n):
        for j in range(i + 1, n):
            if is_connected[i][j]:
                uf.union(i, j)

    provinces = set()
    for i in range(n):
        provinces.add(uf.find(i))

    return len(provinces)
```

Interview Question: When to use Segment Tree vs Fenwick Tree?

Answer: Use Fenwick Tree for simpler range queries and point updates (simpler code, less memory). Use Segment Tree for more complex operations like range updates, range minimum/maximum queries, or when you need lazy propagation.

Chapter 16: Interview Preparation

This chapter contains commonly asked interview questions and their solutions. Practice these thoroughly to ace your technical interviews.

16.1 Must-Know Problems

Q1: Two Sum

Given an array and a target, find two numbers that add up to the target.

Python

```
def two_sum(nums, target):
    map = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in map:
            return [map[complement], i]
        map[num] = i
    return []
# Time: O(n), Space: O(n)
```

Q2: Best Time to Buy and Sell Stock

Python

```
def max_profit(prices):
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        min_price = min(min_price, price)
        max_profit = max(max_profit, price - min_price)
    return max_profit
# Time: O(n), Space: O(1)
```

Q3: Product of Array Except Self

Python

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
```

```
    # Left products
    for i in range(1, n):
        result[i] = result[i-1] * nums[i-1]

    # Right products
    right = 1
    for i in range(n-1, -1, -1):
        result[i] *= right
        right *= nums[i]

    return result
# Time: O(n), Space: O(1) excluding output
```

Q4: Reverse Linked List

Python

```
def reverse_list(head):
    prev = None
    curr = head
    while curr:
        next_node = curr.next
        curr.next = prev
        prev = curr
        curr = next_node
    return prev
# Time: O(n), Space: O(1)
```

Q5: Maximum Depth of Binary Tree

Python

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
# Time: O(n), Space: O(h) where h is height
```

16.2 Tips for Technical Interviews

Before the Interview

- Practice coding on whiteboard or paper
- Review time complexities of common operations

- Understand the problem before jumping to solution
- Practice explaining your thought process

During the Interview

- Clarify the problem and constraints
- Start with a brute force solution
- Optimize step by step
- Test your solution with examples
- Discuss time and space complexity

Common Mistakes to Avoid

- Not asking clarifying questions
- Jumping into code without planning
- Ignoring edge cases
- Not testing the solution

16.3 Quick Reference: Time Complexities

Data Structure	Access	Search
Array	O(1)	O(n)
Linked List	O(n)	O(n)
Binary Search Tree	O(log n)	O(log n)
Hash Table	N/A	O(1)
Heap	O(1) max/min	O(n)

Final Words

Remember that mastering Data Structures and Algorithms is a journey, not a destination. Keep practicing, stay curious, and never stop learning. The concepts in this book form the foundation of computer science and will serve you throughout your career.

Good luck with your interviews and happy coding!

- Mohammad Faiz

Appendix A: Language Syntax Reference

Quick reference guide for common operations across C++, Java, and Python.

A.1 Variable Declaration

C++

```
int x = 10;
float pi = 3.14;
string name = "Hello";
bool flag = true;
```

Java

```
int x = 10;
float pi = 3.14f;
String name = "Hello";
boolean flag = true;
```

Python

```
x = 10  # Integer
pi = 3.14  # Float
name = "Hello"  # String
flag = True  # Boolean
```

A.2 Arrays/Lists

C++

```
int arr[] = {1, 2, 3};
vector<int> vec = {1, 2, 3};
vec.push_back(4);  // Add element
int n = vec.size();  // Get size
```

Java

```
int[] arr = {1, 2, 3};
ArrayList<Integer> list = new ArrayList<>();
list.add(4);  // Add element
int n = list.size();  // Get size
```

Python

```
arr = [1, 2, 3]
arr.append(4)  # Add element
```

```
n = len(arr)  # Get size
# List comprehension
squares = [x**2 for x in range(5)]
```

A.3 Loops

C++

```
// For loop
for (int i = 0; i < n; i++) { ... }

// Range-based for loop
for (int x : arr) { ... }

// While loop
while (condition) { ... }
```

Java

```
// For loop
for (int i = 0; i < n; i++) { ... }

// Enhanced for loop
for (int x : arr) { ... }

// While loop
while (condition) { ... }
```

Python

```
# For loop
for i in range(n):
    ...

# Iterate over list
for x in arr:
    ...

# While loop
while condition:
    ...
```

A.4 Functions/Methods

C++

```
int add(int a, int b) {
    return a + b;
}

// Function with default parameter
void greet(string name = "World") { ... }
```

Java

```
int add(int a, int b) {
    return a + b;
}

// Method overloading
int add(int a, int b) { ... }
double add(double a, double b) { ... }
```

Python

```
def add(a, b):
    return a + b

# Default parameter
def greet(name="World"):
    ...

# Lambda function
square = lambda x: x**2
```

A.5 Common Built-in Functions

Operation	C++	Java	Python
Array length	arr.size()	arr.length	len(arr)
Add to list	push_back()	add()	append()
Sort	sort()	Arrays.sort()	sort() / sorted()
Max element	*max_element()	Collections.max()	max()
Min element	*min_element()	Collections.min()	min()

Appendix B: DSA Cheat Sheet

Quick reference for time and space complexities of common data structures and algorithms.

B.1 Array Operations

```
Access:     O(1)
Search:     O(n)
Insert:     O(n) - shifting required
Delete:     O(n) - shifting required
Append:     O(1) amortized
```

B.2 Linked List Operations

```
Access:     O(n)
Search:     O(n)
Insert:     O(1) - if position known
Delete:     O(1) - if position known
Append:     O(n) - need to traverse
```

B.3 Binary Search Tree

```
Access:     O(log n) average, O(n) worst
Search:     O(log n) average, O(n) worst
Insert:     O(log n) average, O(n) worst
Delete:     O(log n) average, O(n) worst
```

B.4 Hash Table

```
Search:     O(1) average, O(n) worst
Insert:     O(1) average, O(n) worst
Delete:     O(1) average, O(n) worst
```

B.5 Sorting Algorithms Summary

```
Algorithm      | Time (Best/Avg/Worst) | Space
---------------|----------------------|------
Bubble Sort    | O(n) / O(n²) / O(n²)   | O(1)
Selection Sort | O(n²) / O(n²) / O(n²) | O(1)
Insertion Sort | O(n) / O(n²) / O(n²)   | O(1)
Merge Sort     | O(n log n) - all cases | O(n)
```

```
Quick Sort     | O(n log n) / O(n log n) / O(n²) | O(log n)
Heap Sort      | O(n log n) - all cases | O(1)
```

B.6 When to Use What

- Use Arrays when: You need fast access by index
- Use Linked Lists when: You need frequent insertions/deletions
- Use Hash Maps when: You need fast lookups
- Use Trees when: You need ordered data and fast search
- Use Heaps when: You need priority queue operations
- Use Graphs when: You need to model relationships/networks

B.7 Problem-Solving Patterns

- Two Pointers: Sorted arrays, finding pairs
- Sliding Window: Subarray/substring problems
- Fast & Slow Pointers: Cycle detection
- Merge Intervals: Overlapping interval problems
- BFS: Shortest path in unweighted graph
- DFS: Exploring all paths, cycle detection
- DP: Optimization problems with overlapping subproblems
- Backtracking: Permutations, combinations, constraint satisfaction

Multi-Language Edition

C/C++ | Java | Python | JavaScript

Mohammad Faiz

© 2026 All Rights Reserved